PROJECT DOCUMENTATION

Candidate Communication Platform

A full-stack recruiter platform for communicating with candidates and tracking the complete communication journey — from send to read.

Next.js 15 | React 19 | TypeScript | Material UI v7 | Django 4.2 | Django REST Framework | PostgreSQL | Clean Architecture | SOLID Principles

BACKEND	FRONTEND	DATABASE	ARCHITECTURE
Python / Django 4.2	Next.js 15 / TypeScript	PostgreSQL	Clean + SOLID

1 Tech Stack

Layer	Technology	Purpose
Frontend	Next.js 15, React 19, TypeScript, Material UI v7	Server-rendered recruiter dashboard
Backend	Python 3.10+, Django 4.2, Django REST Framework 3.15	REST API, business logic, data persistence
Database	PostgreSQL (primary) + SQLite (test fallback)	Relational storage for candidates & messages
API Docs	drf-spectacular (Swagger / OpenAPI 3.0)	Auto-generated interactive API documentation
Provider	Dummy microservice (Flask, port 9000)	Simulates SMS / WhatsApp / Email sending & webhooks
Serving	gunicorn + WhiteNoise	Production-ready static file & WSGI serving
Architecture	Clean Architecture + SOLID Principles	Layered, testable, maintainable codebase

2 Project Structure

```
Candidate Communication Platform/
├── frontend/                        # Next.js + TypeScript + MUI
│   └── src/
│       ├── app/                     # Next.js App Router pages
│       ├── components/              # 13 React components
│       │   ├── CandidateDashboard.tsx
│       │   ├── CandidateTable.tsx
│       │   ├── AddCandidateModal.tsx
│       │   ├── SendMessageModal.tsx
│       │   ├── BulkMessageModal.tsx
│       │   ├── CandidateDetailModal.tsx
│       │   ├── TemplatesDrawer.tsx
│       │   ├── ServiceHealthBar.tsx
│       │   └── ...
│       ├── context/                 # React context providers (Toast)
│       ├── lib/                     # API client (candidatesApi, communicationsApi)
│       └── types/                   # TypeScript type definitions
│
├── backend/                         # Django REST API
│   ├── core/                        # Django project settings & root URLs
│   ├── apps/
│   │   ├── candidates/              # Candidate CRUD
│   │   │   ├── domain/              # entities.py, repositories.py
│   │   │   ├── application/         # services.py (use cases)
│   │   │   ├── infrastructure/      # ORM models + repository impl
│   │   │   └── api/                 # DRF serializers & views
│   │   └── communications/          # Messaging engine
│   │       ├── domain.py            # CommunicationProvider ABC, DTOs
│   │       ├── services.py          # CommunicationService (672 lines)
│   │       ├── providers.py         # DummyServiceProvider + MockProvider
│   │       ├── failures.py          # Failure translation layer
│   │       ├── templates.py         # Template placeholder rendering
│   │       ├── models.py            # Message, MessageTemplate, MessageStatusLog
│   │       ├── serializers.py       # DRF serializers
│   │       ├── views.py             # API Views (416 lines)
│   │       ├── urls.py              # 13 URL routes
│   │       └── tests.py             # 508-line test suite
│   └── requirements.txt
│
└── dummy_provider/
    └── server.py                    # Standalone Flask microservice (port 9000)
```

3 Architecture Decisions

3.1 Clean Architecture (Backend)

The backend is organised into four strict, dependency-directed layers. No outer layer is imported by an inner layer.



3.2 SOLID Principles Applied

S **Single Responsibility**

`CandidateService` handles only CRUD use-cases; `CommunicationService` handles only sending & status tracking; `CommunicationFailureTranslator` handles only error humanisation.

O **Open / Closed**

New communication channels (e.g. Telegram) can be added by implementing `CommunicationProvider` ABC and registering in `ProviderFactory` — zero existing-code changes required.

L **Liskov Substitution**

`DummyServiceCommunicationProvider`, `InMemoryMockProvider`, and test `StubFailProvider` are all interchangeable anywhere `CommunicationProvider` is expected. Tests exploit this directly.

I **Interface Segregation**

`CommunicationProvider` exposes only three methods: `send()`, `check_status()`, `health()`. Candidate repository interface is similarly narrow.

D **Dependency Inversion**

`CommunicationService.__init__` accepts an optional `CommunicationProvider` abstraction — never a concrete class. Tests inject stubs; production uses `ProviderFactory`.

3.3 Provider Abstraction

A standalone **dummy provider microservice** (`dummy_provider/server.py`) runs on port 9000 and simulates a real communication vendor with realistic behaviours:

✅

Normal Success

Returns a `reference_id` ; fires Sent → Delivered → Read webhooks over time.

❌

Delivery Failure

Simulates carrier rejection / invalid recipient (HTTP 422).

⏱️

Rate Limiting

Simulates HTTP 429 burst throttle; classified as retryable.

🔌

Temporary Outage

Simulates HTTP 503; automatically becomes retryable on retry.

3.4 Idempotent Status Progression

Message status follows a strict one-way FSM. The service refuses to regress a status backwards:

```
STATUS_PRECEDENCE = {
  "scheduled" / "pending": 0,
  "sent": 1,
  "delivered": 2,
  "read": 3
}
# Incoming webhook/poll ignored if new_precedence ⩽ current_precedence
```

This means duplicate webhook deliveries and out-of-order events are safely discarded.

3.5 Failure Classification & Humanisation

`CommunicationFailureTranslator` converts raw provider HTTP codes and exception messages into normalised `FailureCategory` enums and human-readable recruiter-facing strings.

Category	Retryable	Example Trigger
`INVALID_RECIPIENT`	No	Bad phone number / email
`RATE_LIMIT_EXCEEDED`	Yes	HTTP 429
`TEMPORARY_PROVIDER_FAILURE`	Yes	HTTP 502/503/504
`DELIVERY_FAILURE`	Yes	Carrier rejection
`SERVICE_UNAVAILABLE`	Yes	Provider timeout
`OTHER_PROVIDER_ERROR`	No	Unknown error

3.6 Frontend Architecture

The Next.js frontend uses the **App Router** pattern with a single-page dashboard UX. All server communication is abstracted through a typed API client in `src/lib/api.ts`.



CandidateDashboard

Root orchestrator component; manages all state, pagination, sort, and modal visibility.



CandidateTable

Sortable, paginated table with inline status badges and row-action menu.



SendMessageModal

Individual message sending — channel picker, template selector, live preview.



BulkMessageModal

Bulk messaging to multiple selected candidates with per-candidate outcome display.

4 Setup Instructions

Prerequisites: Python 3.10+, PostgreSQL, Node.js 18+, and npm must be installed and available on your PATH.

4.1 Backend Setup

1 Create & activate virtual environment

```
cd backend
python -m venv venv
# Windows:
venv\Scripts\activate
# macOS/Linux:
source venv/bin/activate
```

2 Install dependencies

```
pip install -r requirements.txt
```

3 Configure environment

```
copy .env.example .env   # Windows
cp .env.example .env     # macOS/Linux
# Edit .env — set DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT
# Also set DUMMY_PROVIDER_URL=http://localhost:9000
```

4 Create PostgreSQL database

```
psql -U postgres
CREATE DATABASE candidate_comm_db;
```

5 Run migrations

```
python manage.py makemigrations candidates communications
python manage.py migrate
```

6 **(Optional) Create superuser for admin panel**

```
python manage.py createsuperuser
```

7 **Start the backend server**

```
python manage.py runserver
# API: http://localhost:8000
# Swagger: http://localhost:8000/api/docs/
# Admin: http://localhost:8000/admin/
```

4.2 Dummy Provider Setup

1 **Install provider dependencies & start**

```
cd dummy_provider
pip install -r requirements.txt
python server.py
# Provider API running at: http://localhost:9000
```

4.3 Frontend Setup

1 **Install Node dependencies**

```
cd frontend
npm install
```

2 **Configure environment**

```
# .env.local (already present)
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
```

3 Start the dev server

```
npm run dev
# App: http://localhost:3000
```

> ✅ All three processes (**backend**, **dummy_provider**, **frontend**) must run concurrently for full functionality.

4.4 Running Tests

```
cd backend
python manage.py test apps.candidates apps.communications --verbosity=2
```

5 API Reference

5.1 Candidates

Method	Endpoint	Description
GET	`/api/v1/candidates/`	List candidates (paginated, searchable by name / phone / email / code)
POST	`/api/v1/candidates/`	Create a new candidate
GET	`/api/v1/candidates/{id}/`	Retrieve a single candidate
PATCH	`/api/v1/candidates/{id}/`	Partial update of candidate details
DELETE	`/api/v1/candidates/{id}/`	Delete a candidate
POST	`/api/v1/candidates/bulk-delete/`	Delete multiple candidates at once

5.2 Communications

Method	Endpoint	Description
POST	`/api/v1/communications/send/`	Send a message to a single candidate
POST	`/api/v1/communications/bulk-send/`	Send same message to multiple candidates (independent tracking)
GET	`/api/v1/communications/candidate/{id}/`	Retrieve full message history for a candidate
GET	`/api/v1/communications/messages/{id}/`	Retrieve message detail & current status
POST	`/api/v1/communications/messages/{id}/sync/`	Force-sync status from provider
POST	`/api/v1/communications/messages/{id}/retry/`	Retry a failed message
POST	`/api/v1/communications/webhook/`	Receive provider webhook status updates (idempotent)

Method	Endpoint	Description
GET	`/api/v1/communications/providers/`	List available providers & their health
GET	`/api/v1/communications/health/`	Provider health check
POST	`/api/v1/communications/dispatch-scheduled/`	Dispatch due scheduled messages
GET	`/api/v1/communications/templates/`	List / create message templates
POST	`/api/v1/communications/templates/preview/`	Preview a template with candidate placeholders
GET	`/api/v1/communications/templates/{id}/`	Retrieve / update / delete a specific template

6 Assumptions Made

Authentication is out of scope

The platform does not implement user login or JWT tokens. All API endpoints are publicly accessible as per typical take-home project scope.

Dummy provider simulates a real vendor

The `dummy_provider/server.py` standalone Flask app stands in for Twilio / SendGrid. It fires realistic webhook callbacks (Sent → Delivered → Read) and simulates failure modes.

Message content is free-form text

No character limits or encoding constraints are enforced beyond what the provider accepts. Templates use `{{placeholder}}` syntax for personalisation.

Candidate status reflects the last message outcome

A candidate's overall status (NA / Sent / Delivered / Read / Failed / Scheduled) reflects the most recent message sent, not an aggregate.

Scheduled messages require a manual dispatch trigger

Scheduled messages are stored and dispatched via `POST /dispatch-scheduled/`. A real deployment would use Celery Beat or a cron job; for the project scope, manual or automated polling is assumed.

One candidate, one mobile number

Candidates have a single `mobile_number` field used for both SMS and WhatsApp channels.

Candidate IDs are system-generated

The `candidate_id` is an auto-generated alphanumeric code (e.g. `A1B2C3`) set by the ORM on creation — not user-provided.

7 Completed Features

Core Requirements

- ✓ **Candidate Management (CRUD)**
  Add, view, edit, delete candidates. Searchable by name, phone, email, and candidate code. Auto-generated alphanumeric candidate ID.

- ✓ **Individual Message Sending**
  Send SMS, WhatsApp, or Email to a single candidate via any configured provider. Content or template-based messages.

- ✓ **Bulk Messaging**
  Select multiple candidates and send the same message to all at once. Each candidate's message is tracked independently — one failure does not abort the rest.

- ✓ **Message Status Tracking with Idempotent Updates**
  Messages progress through Scheduled → Pending → Sent → Delivered → Read (strict one-way FSM). Duplicate or out-of-order webhook events are safely rejected.

- ✓ **Communication Failure Translation**
  Raw provider errors (HTTP codes, exception messages) are classified into recruiter-friendly categories with actionable descriptions and retryable flags.

- ✓ **Candidate Communication History**
  Full message timeline per candidate, including channel, status, provider, timestamps, and error details for failed messages.

- ✓ **Webhook Receiver**
  Accepts provider status update callbacks. Idempotent — duplicate webhooks for the same status are silently ignored.

- ✓ **Provider Health Monitoring**
  Backend polls the dummy provider's `/health` endpoint. The frontend displays a live `ServiceHealthBar` showing provider status.

- ✓ **Dummy Provider Microservice**
  Standalone Flask service that simulates a real communication vendor, including success, failure, rate-limit, and outage scenarios with realistic webhook firing.

- ✓ **OpenAPI / Swagger Documentation**
  Auto-generated interactive API docs via `drf-spectacular` at `/api/docs/`.

- ✓ **Comprehensive Test Suite**

558 lines of unit and integration tests covering Requirement #7 (idempotent status updates), #8 (failure translation), and #10 (communication history). LSP verified via stub providers.

Bonus Features

- **Message Templates with Personalisation**
  Create reusable templates with `{{full_name}}`, `{{candidate_id}}`, `{{mobile_number}}`, `{{email}}` placeholders. Templates are substituted at send time per candidate. CRUD API + `TemplatesDrawer` UI.

- **Message Scheduling**
  Messages can be scheduled for future delivery by passing a `scheduled_at` datetime. A dispatch endpoint sends all due scheduled messages and updates their status.

- **Message Retry**
  Failed messages with retryable failure categories can be retried via `POST /messages/{id}/retry/`. The retry uses the same provider, channel, and content.

- **Provider Status Sync**
  Force-poll provider for the latest delivery status via `POST /messages/{id}/sync/` — useful when webhooks are delayed or missed.

- **Bulk Delete**
  Select and delete multiple candidates in a single API call (`POST /bulk-delete/`) with confirmation dialog on the frontend.

- **Sortable Candidate Table**
  Candidates can be sorted by name, phone, last message date, status, or candidate ID in ascending/descending order.

- **Toast Notification System**
  Global React context (`ToastContext`) drives non-blocking success/error toasts across all modals and actions.

8 Key Data Models

Candidate

```
class Candidate(models.Model):
    candidate_id      = CharField(max_length=10, unique=True)  # e.g. A1B2C3
    full_name         = CharField(max_length=200)
    mobile_number     = CharField(max_length=20, unique=True)
    email             = EmailField(blank=True)
    status            = CharField(choices=MessageStatus)       # NA/sent/delivered/read/fail
    last_message_date = DateField(null=True)
```

Message

```
class Message(models.Model):
    candidate          = ForeignKey(Candidate)
    channel            = CharField(choices=['sms','whatsapp','email'])
    content            = TextField()
    status             = CharField(choices=MessageStatusLog)
    provider           = CharField()                            # e.g. "dummy-service"
    provider_message_id= CharField()
    failure_category   = CharField()                            # FailureCategory enum
    error_message      = TextField()
    scheduled_at       = DateTimeField(null=True)
    template           = ForeignKey(MessageTemplate, null=True)
    retry_count        = PositiveIntegerField(default=0)
```

MessageTemplate

```
class MessageTemplate(models.Model):
    name       = CharField(max_length=120)
    content    = TextField()              # supports {{full_name}}, {{candidate_id}}, etc.
    channel    = CharField(blank=True)    # optional channel affinity
```

Django REST Framework · Next.js 15 · PostgreSQL · TypeScript · Material UI